

KUBERNETES-TASK-04

1. Deploy an application using a Deployment with 3 replicas and a rolling update strategy.

What is a Deployment?

A Deployment manages ReplicaSets and Pods.

It provides:

- Desired number of replicas
- Rolling updates
- Rollbacks
- Self-healing
- Version management

Step 1: Create YAML

```
ubuntu@k8s-master:~$ vi nginx-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

Save and exit.

Step 2: Deploy:

Step 3: Check Deployment:

Step 4: Check ReplicaSet:

```
ubuntu@k8s-master:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
ubuntu@k8s-master:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    3/3     3            3           20s
ubuntu@k8s-master:~$ kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
nginx-deployment-7c977bd7bb         3         3         3       36s
```

Step 5: Check Pods:

```
ubuntu@k8s-master:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-deployment-7c977bd7bb-bmplk	1/1	Running	0	51s	10.244.1.30	worker01	<none>	<none>
nginx-deployment-7c977bd7bb-ng8ds	1/1	Running	0	51s	10.244.2.28	worker02	<none>	<none>
nginx-deployment-7c977bd7bb-wgds7	1/1	Running	0	51s	10.244.1.31	worker01	<none>	<none>

What is Rolling Update?

Suppose your current version is:

nginx:1.27

You want to update to:

nginx:1.28

Kubernetes does not normally delete all old Pods at once.

Update image

kubectl set image deployment/nginx-deployment nginx=nginx:1.28

Watch the update:

kubectl rollout status deployment/nginx-deployment

```
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-deployment nginx=nginx:1.28
deployment.apps/nginx-deployment image updated
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-deployment
deployment "nginx-deployment" successfully rolled out
ubuntu@k8s-master:~$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-859f76c7fc-9kjt5	1/1	Running	0	24s
nginx-deployment-859f76c7fc-jjpzs	1/1	Running	0	20s
nginx-deployment-859f76c7fc-rkc65	1/1	Running	0	24s

Use this cmd: kubectl describe deployment

```
Conditions:
  Type           Status  Reason
  ----           -
  Available       True    MinimumReplicasAvailable
  Progressing     True    NewReplicaSetAvailable
OldReplicaSets:  nginx-deployment-7c977bd7bb (0/0 replicas created)
NewReplicaSet:   nginx-deployment-859f76c7fc (3/3 replicas created)
Events:
  Type    Reason              Age   From          Message
  ----    -
  Normal  ScalingReplicaSet   20m   deployment-controller  Scaled up replica set nginx-deployment-7c977bd7bb to 3
  Normal  ScalingReplicaSet   18m   deployment-controller  Scaled up replica set nginx-deployment-859f76c7fc to 1
  Normal  ScalingReplicaSet   18m   deployment-controller  Scaled down replica set nginx-deployment-7c977bd7bb to 2 from 3
  Normal  ScalingReplicaSet   18m   deployment-controller  Scaled up replica set nginx-deployment-859f76c7fc to 2 from 1
  Normal  ScalingReplicaSet   18m   deployment-controller  Scaled down replica set nginx-deployment-7c977bd7bb to 1 from 2
  Normal  ScalingReplicaSet   18m   deployment-controller  Scaled up replica set nginx-deployment-859f76c7fc to 3 from 2
  Normal  ScalingReplicaSet   18m   deployment-controller  Scaled down replica set nginx-deployment-7c977bd7bb to 0 from 1
ubuntu@k8s-master:~$
```

2. Configure a Deployment with a Recreate strategy and observe the downtime.

A Recreate strategy deletes the existing Pods before creating the new Pods.

Therefore, there can be downtime.

Create:

```
ubuntu@k8s-master:~$ vi recreate-deployment.yaml
ubuntu@k8s-master:~$
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-recreate
spec:
  replicas: 3
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: nginx-recreate
  template:
    metadata:
      labels:
        app: nginx-recreate
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80

```

Apply & check the pods:

Next update:

```

ubuntu@k8s-master:~$ kubectl apply -f recreate-deployment.yaml
deployment.apps/nginx-recreate created
ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-859f76c7fc-9kjt5   1/1     Running   0           73m
nginx-deployment-859f76c7fc-jjpzs   1/1     Running   0           73m
nginx-deployment-859f76c7fc-rkc65   1/1     Running   0           73m
nginx-recreate-6cc7489b-6brmz       1/1     Running   0           106s
nginx-recreate-6cc7489b-tf66l       1/1     Running   0           106s
nginx-recreate-6cc7489b-wp979       1/1     Running   0           106s
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-recreate nginx=nginx:1.28
deployment.apps/nginx-recreate image updated

```

You can observe that the old Pods are terminated before the new Pods are created.

```

Conditions:
  Type           Status  Reason
  ----           -
  Available       True    MinimumReplicasAvailable
  Progressing     True    NewReplicaSetAvailable
OldReplicaSets:  nginx-recreate-6cc7489b (0/0 replicas created)
NewReplicaSet:   nginx-recreate-5f7b9bbc99 (3/3 replicas created)
Events:
  Type    Reason              Age   From                  Message
  ----    -
  Normal  ScalingReplicaSet   10m   deployment-controller  Scaled up replica set nginx-recreate-6cc7489b to 3
  Normal  ScalingReplicaSet   7m58s deployment-controller  Scaled down replica set nginx-recreate-6cc7489b to 0 from 3
  Normal  ScalingReplicaSet   7m57s deployment-controller  Scaled up replica set nginx-recreate-5f7b9bbc99 to 3
ubuntu@k8s-master:~$

```

RollingUpdate vs Recreate

RollingUpdate	Recreate
Gradually replaces Pods	Deletes old Pods first
Usually no downtime	Causes downtime
Default strategy	Must explicitly configure
Good for most web applications	Useful when old/new versions cannot run together

3. Update an existing Deployment and perform a rollback to the previous version.

This is one of the most important interview topics.

Now update the image:

Check pods:

Check history:

```
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-recreate nginx=nginx:1.28
ubuntu@k8s-master:~$ kubectl get deployment nginx-recreate \
-o=jsonpath='{.spec.template.spec.containers[0].image}'
nginx:1.28ubuntu@k8s-master:~$ kubectl get deployment nginx-recreate \recreate \
-o=jsonpath='{.spec.template.spec.containers[0].image}'
nginx:1.28ubuntu@k8s-master:~$ kubectl get pods -w
NAME                                READY    STATUS    RESTARTS   AGE
nginx-deployment-859f76c7fc-9kjt5    1/1      Running   0           109m
nginx-deployment-859f76c7fc-jjpzs    1/1      Running   0           109m
nginx-deployment-859f76c7fc-rkc65    1/1      Running   0           109m
nginx-recreate-5f7b9bbc99-5fgmb      1/1      Running   0           35m
nginx-recreate-5f7b9bbc99-96mb7      1/1      Running   0           35m
nginx-recreate-5f7b9bbc99-rh76x      1/1      Running   0           35m
^Cubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-recreate
deployment "nginx-recreate" successfully rolled out
```

If you want to **observe the downtime live**, we need to trigger another image change while watching.

I check this in another terminal:

```
^Cubuntu@k8s-master:~$ kubectl get deployment nginx-recreate -w
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
nginx-recreate                      3/3      3              3            33m
^Cubuntu@k8s-master:~$ kubectl get deployment nginx-recreate -w
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
nginx-recreate                      3/3      3              3            40m
nginx-recreate                      3/3      3              3            40m
nginx-recreate                      3/3      3              3            40m
nginx-recreate                      3/3      0              3            40m
nginx-recreate                      0/3      0              0            40m
nginx-recreate                      0/3      0              0            40m
nginx-recreate                      0/3      3              0            40m
nginx-recreate                      1/3      3              1            40m
nginx-recreate                      2/3      3              2            40m
nginx-recreate                      3/3      3              3            40m
```

Now suppose the new version has an issue.

Perform rollback:

```
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

ubuntu@k8s-master:~$ kubectl rollout history deployment/nginx-deployment --revision=1
deployment.apps/nginx-deployment with revision #1
Pod Template:
  Labels:      app=nginx
              pod-template-hash=7c977bd7bb
  Containers:
    nginx:
      Image:      nginx:1.27
      Port:       80/TCP
      Host Port:  0/TCP
      Environment: <none>
      Mounts:      <none>
      Volumes:     <none>

ubuntu@k8s-master:~$ kubectl rollout history deployment/nginx-deployment --revision=2
deployment.apps/nginx-deployment with revision #2
Pod Template:
  Labels:      app=nginx
              pod-template-hash=859f76c7fc
  Containers:
    nginx:
      Image:      nginx:1.28
      Port:       80/TCP
      Host Port:  0/TCP
      Environment: <none>
      Mounts:      <none>
      Volumes:     <none>
```

```
ubuntu@k8s-master:~$ kubectl rollout undo deployment/nginx-deployment --to-revision=1
deployment.apps/nginx-deployment rolled back
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-deployment
deployment "nginx-deployment" successfully rolled out
```

After rollback, Kubernetes may create another revision. So don't expect the history to simply go back to only revision 1.

```
nginx:1.27ubuntu@k8s-master:~$ kubectl rollout history deployment/nginx-deployment
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
2         <none>
3         <none>
```

Rollback means reverting a Deployment to a previous stable ReplicaSet or revision when the new version has issues.

4. Modify a Deployment to add resource requests and limits for CPU and memory.

Resource requests and limits are very important for Kubernetes scheduling.

Create: YAML file

Apply:

```
ubuntu@k8s-master:~$ vi resource-deployment.yaml
ubuntu@k8s-master:~$ kubectl apply -f resource-deployment.yaml
deployment.apps/resource-deployment created
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: resource-nginx
  template:
    metadata:
      labels:
        app: resource-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "200m"
              memory: "256Mi"
```

Check pods:

```
ubuntu@k8s-master:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-deployment-7c977bd7bb-9n4nr	1/1	Running	0	28m	10.244.1.37	worker01	<none>	<none>
nginx-deployment-7c977bd7bb-w5qtm	1/1	Running	0	28m	10.244.1.38	worker01	<none>	<none>
nginx-deployment-7c977bd7bb-wr5gg	1/1	Running	0	28m	10.244.2.36	worker02	<none>	<none>
nginx-recreate-6cc7489b-f5tfr	1/1	Running	0	34m	10.244.2.35	worker02	<none>	<none>
nginx-recreate-6cc7489b-hrrdv	1/1	Running	0	34m	10.244.2.34	worker02	<none>	<none>
nginx-recreate-6cc7489b-k5p8f	1/1	Running	0	34m	10.244.1.36	worker01	<none>	<none>
resource-deployment-7b45bcd668-9m9p2	1/1	Running	0	49s	10.244.1.40	worker01	<none>	<none>
resource-deployment-7b45bcd668-qqltx	1/1	Running	0	49s	10.244.1.39	worker01	<none>	<none>
resource-deployment-7b45bcd668-ztgkb	1/1	Running	0	49s	10.244.2.37	worker02	<none>	<none>

Check resources:

```
ubuntu@k8s-master:~$ kubectl describe pod resource-deployment-7b45bcd668-9m9p2
Name:          resource-deployment-7b45bcd668-9m9p2
Namespace:     default
Priority:       0
Service Account: default
Node:          worker01/172.31.4.11
Start Time:    Mon, 28 Sep 2026 13:13:08 +0000
Labels:        app=resource-nginx
               pod-template-hash=7b45bcd668
Annotations:   <none>
Status:        Running
IP:            10.244.1.40
IPs:           10.244.1.40
Controlled By: ReplicaSet/resource-deployment-7b45bcd668
Containers:
  nginx:
    Container ID:   containerd://69e33836454cf8d9051df431fe6624e87a2b59bc848cf6327583f21ffdlcedbb
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 28 Sep 2026 13:13:09 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:          200m
      memory:       256Mi
    Requests:
      cpu:          100m
      memory:       128Mi
    Environment:    <none>
    Mounts:          <none>
```

Requests vs Limits

Request

Request means:

"Kubernetes, I need at least this amount of resources."

Example:

requests:

cpu: "100m"

memory: "128Mi"

The scheduler uses **requests** when deciding which node can run the Pod.

Limit

Limit means:

"The container should not use more than this amount."

Example:

limits:

cpu: "200m"

memory: "256Mi"

Easy interview explanation

Requests are used by the scheduler for Pod placement, while limits define the maximum resources the container can consume.

5. Create a Deployment with MaxSurge and MaxUnavailable configurations.

These two settings control how a RollingUpdate happens.

Create: YAML file

vi surge-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: surge-deployment

spec:
  replicas: 4

  strategy:
    type: RollingUpdate

    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1

  selector:
    matchLabels:
      app: surge-nginx

  template:
    metadata:
      labels:
        app: surge-nginx

    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

Apply:

```
Last login: Mon Sep 28 13:08:51 2026 from 18.206.107.29
ubuntu@k8s-master:~$ vi surge-deployment.yaml
ubuntu@k8s-master:~$ vi surge-deployment.yaml
ubuntu@k8s-master:~$ kubectl apply -f surge-deployment.yaml
deployment.apps/surge-deployment created
ubuntu@k8s-master:~$
```

Check pods:

Now update:

kubectl set image deployment/surge-deployment nginx=nginx:1.28

Watch:

kubectl get pods -w

```

ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c977bd7bb-9n4nr   1/1     Running   0           44m
nginx-deployment-7c977bd7bb-w5qtm   1/1     Running   0           44m
nginx-deployment-7c977bd7bb-wr5gg   1/1     Running   0           44m
nginx-recreate-6cc7489b-f5tfr       1/1     Running   0           51m
nginx-recreate-6cc7489b-hrtdv       1/1     Running   0           51m
nginx-recreate-6cc7489b-k5p8f       1/1     Running   0           51m
resource-deployment-7b45bcd668-9m9p2 1/1     Running   0           17m
resource-deployment-7b45bcd668-qgltx 1/1     Running   0           17m
resource-deployment-7b45bcd668-ztgkb 1/1     Running   0           17m
surge-deployment-66cc69b458-84qgd    1/1     Running   0           53s
surge-deployment-66cc69b458-92k7v    1/1     Running   0           53s
surge-deployment-66cc69b458-98m8f    1/1     Running   0           53s
surge-deployment-66cc69b458-cs5x9    1/1     Running   0           53s
ubuntu@k8s-master:~$ kubectl set image deployment/surge-deployment nginx=nginx:1.28
deployment.apps/surge-deployment image updated
ubuntu@k8s-master:~$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c977bd7bb-9n4nr   1/1     Running   0           45m
nginx-deployment-7c977bd7bb-w5qtm   1/1     Running   0           45m
nginx-deployment-7c977bd7bb-wr5gg   1/1     Running   0           45m
nginx-recreate-6cc7489b-f5tfr       1/1     Running   0           51m
nginx-recreate-6cc7489b-hrtdv       1/1     Running   0           51m
nginx-recreate-6cc7489b-k5p8f       1/1     Running   0           51m
resource-deployment-7b45bcd668-9m9p2 1/1     Running   0           17m
resource-deployment-7b45bcd668-qgltx 1/1     Running   0           17m
resource-deployment-7b45bcd668-ztgkb 1/1     Running   0           17m
surge-deployment-5b4dcdc97c-7c542    1/1     Running   0           7s
surge-deployment-5b4dcdc97c-84qcz    1/1     Running   0           9s
surge-deployment-5b4dcdc97c-h76wm    1/1     Running   0           9s
surge-deployment-5b4dcdc97c-vndqm    1/1     Running   0           8s

```

What is MaxSurge?

You have:

replicas: 4

maxSurge: 1

Normally:

Desired = 4

During update Kubernetes can temporarily create:

4 + 1 = 5 Pods

So:

maxSurge controls how many extra Pods can be created above the desired replica count.

What is MaxUnavailable?

You have:

replicas: 4

maxUnavailable: 1

Kubernetes can have at most one unavailable Pod during the update.

So you can think:

4 desired Pods

1 old Pod can become unavailable

while new Pods are created.

Easy interview answer

MaxSurge controls the maximum number of extra Pods created during a rolling update, while MaxUnavailable controls how many Pods can be unavailable during the update.

6. Set up a Deployment with a custom revision history limit.

Kubernetes keeps old ReplicaSets so that you can perform rollbacks.

You can control how many old revisions are retained.

Create:

vi revision-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: revision-deployment

spec:
  replicas: 3

  revisionHistoryLimit: 3

  selector:
    matchLabels:
      app: revision-nginx

  template:
    metadata:
      labels:
        app: revision-nginx

    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

Apply & Check:

```
ubuntu@k8s-master:~$ vi revision-deployment.yaml
ubuntu@k8s-master:~$ kubectl apply -f revision-deployment.yaml
deployment.apps/revision-deployment created
ubuntu@k8s-master:~$ kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
nginx-deployment-7c977bd7bb         3         3         3       3h26m
nginx-deployment-859f76c7fc         0         0         0       3h23m
nginx-recreate-5f7b9bbc99           0         0         0       129m
nginx-recreate-6cc7489b            3         3         3       131m
resource-deployment-7b45bcd668      3         3         3       57m
revision-deployment-56d7868749      3         3         3       6s
surge-deployment-5b4dcdc97c         4         4         4       39m
surge-deployment-66cc69b458         0         0         0       40m
ubuntu@k8s-master:~$
```

Now perform several updates:

```
ubuntu@k8s-master:~$ kubectl set image deployment/revision-deployment nginx=nginx:1.28
deployment.apps/revision-deployment image updated
ubuntu@k8s-master:~$
```

Then:

```
ubuntu@k8s-master:~$ kubectl set image deployment/revision-deployment nginx=nginx:1.29
deployment.apps/revision-deployment image updated
ubuntu@k8s-master:~$
```

Check:

kubectl get rs

```
ubuntu@k8s-master:~$ kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
nginx-deployment-7c977bd7bb         3         3         3       3h29m
nginx-deployment-859f76c7fc         0         0         0       3h27m
nginx-recreate-5f7b9bbc99           0         0         0       133m
nginx-recreate-6cc7489b             3         3         3       135m
resource-deployment-7b45bcd668      3         3         3       61m
revision-deployment-56d7868749       0         0         0       3m47s
revision-deployment-86744b7cf4       0         0         0       100s
revision-deployment-fd864f9d7        3         3         3       34s
surge-deployment-5b4dcdc97c         4         4         4       43m
surge-deployment-66cc69b458         0         0         0       44m
```

The Deployment retains old ReplicaSets according to:

revisionHistoryLimit: 3

Interview answer

revisionHistoryLimit defines how many old ReplicaSets are retained by the Deployment for rollback purposes.

7. Pause a Deployment during an update, and then resume it.

Kubernetes allows you to pause a Deployment.

This is useful when you want to make multiple changes before allowing the Deployment to complete the rollout.

First create a Deployment:

```
kubectl apply -f nginx-deployment.yaml
```

Pause it:

```
kubectl rollout pause deployment/nginx-deployment
```

Check:

```
kubectl rollout status deployment/nginx-deployment
```

Now make a change:

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.28
```

You can make another change as well.

For example:

```
kubectl label deployment nginx-deployment environment=prod
```

Now resume:

```
kubectl rollout resume deployment/nginx-deployment
```

Check:

```
kubectl rollout status deployment/nginx-deployment
```

And:

```
kubectl get pods
```

```
ubuntu@k8s-master:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
ubuntu@k8s-master:~$ kubectl rollout pause deployment/nginx-deployment
deployment.apps/nginx-deployment paused
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-deployment
deployment "nginx-deployment" successfully rolled out
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-deployment nginx=nginx:1.28
deployment.apps/nginx-deployment image updated
ubuntu@k8s-master:~$ kubectl label deployment nginx-deployment environment=prod
deployment.apps/nginx-deployment labeled
ubuntu@k8s-master:~$ kubectl rollout resume deployment/nginx-deployment
deployment.apps/nginx-deployment resumed
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-deployment
deployment "nginx-deployment" successfully rolled out
ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-859f76c7fc-5hdds    1/1     Running   0          16s
nginx-deployment-859f76c7fc-s7kv2    1/1     Running   0          15s
nginx-deployment-859f76c7fc-skn8x    1/1     Running   0          14s
```

To verify:

```
ubuntu@k8s-master:~$ kubectl get deployment nginx-deployment -o jsonpath='{.spec.template.spec.containers[0].image}'
nginx:1.28
ubuntu@k8s-master:~$ kubectl describe deployment nginx-deployment
Name:          nginx-deployment
Namespace:     default
CreationTimestamp: Mon, 28 Sep 2026 14:10:58 +0000
Labels:        environment=prod
Annotations:    deployment.kubernetes.io/revision: 2
Selector:      app=nginx
Replicas:      3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:  RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 1 max unavailable, 1 max surge
Pod Template:
  Labels:  app=nginx
  Containers:
    nginx:
      Image:        nginx:1.28
      Port:         80/TCP
      Host Port:    0/TCP
      Environment:  <none>
      Mounts:       <none>
      Volumes:      <none>
  Conditions:
    Type           Status  Reason
    ----           -
    Available      True    MinimumReplicasAvailable
    Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  nginx-deployment-7c977bd7bb (0/0 replicas created)
NewReplicaSet:   nginx-deployment-859f76c7fc (3/3 replicas created)
```

Why pause?

kubectl rollout pause temporarily pauses a Deployment rollout. We can make multiple changes and then use kubectl rollout resume to continue the rollout.

8. Create a pod using resource requests for memory and CPU, and observe how the scheduler assigns it to a node.

This task is important because it demonstrates how Kubernetes schedules Pods.

Create:

vi resource-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: nginx1
    image: nginx:1.27
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "200m"
        memory: "256Mi"
  - name: nginx2
    image: nginx:1.27
    command: ["/bin/sh", "-c"]
    args:
    - sed -i 's/listen      80;/listen      8080;/' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "200m"
        memory: "256Mi"
```

Apply:

kubectl apply -f resource-pod.yaml

```
ubuntu@k8s-master:~$ vi resource-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f resource-pod.yaml
pod/resource-pod created
```

Check:

kubectl get pod resource-pod -o wide

```
ubuntu@k8s-master:~$ kubectl get pod resource-pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE   READINESS GATES
resource-pod   2/2     Running   0          7s    10.244.2.54   worker02   <none>            <none>
```

Here:

NODE = worker02

The Kubernetes scheduler selected worker02.

Observe the scheduling event

This is the most useful part for your lab.

Run:

kubectl describe pod resource-pod

Go to the bottom and look at **Events**.

This line proves scheduler assignment:

Successfully assigned default/resource-pod to worker02

```
Events:
  Type     Reason      Age   From              Message
  ----     -
  Normal   Scheduled   5m26s  default-scheduler  Successfully assigned default/resource-pod to worker02
  Normal   Pulled      5m26s  kubelet           Container image "nginx:1.27" already present on machine
  Normal   Created     5m25s  kubelet           Created container: nginx1
  Normal   Started     5m25s  kubelet           Started container nginx1
  Normal   Pulled      5m25s  kubelet           Container image "nginx:1.27" already present on machine
  Normal   Created     5m25s  kubelet           Created container: nginx2
  Normal   Started     5m25s  kubelet           Started container nginx2
ubuntu@k8s-master:~$ kubectl get nodes
NAME           STATUS    ROLES    AGE   VERSION
k8s-master     Ready     control-plane  6d7h  v1.29.15
worker01       Ready     <none>      6d7h  v1.29.15
worker02       Ready     <none>      6d7h  v1.29.15
ubuntu@k8s-master:~$
```

Interview answer:

Kubernetes Scheduler uses the Pod's resource requests to determine which node has enough available resources to run the Pod. After selecting a suitable node, the Pod is assigned to that node.